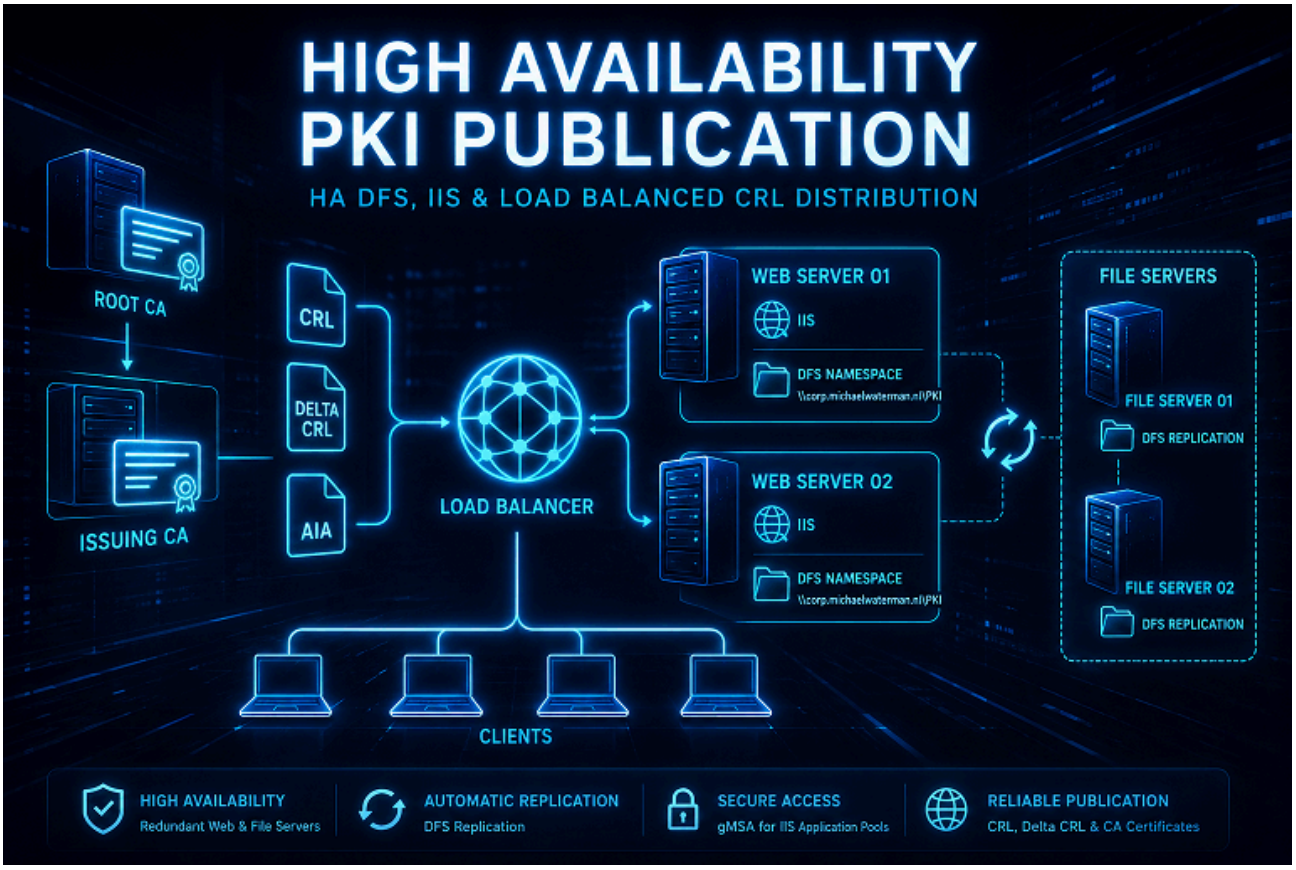
Building a Highly Available CRL and AIA Distribution Platform for AD CS – Michael Waterman

michaelwaterman.nl/2026/06/04/building-a-highly-available-crl-and-aia-distribution-platform-for-ad-cs

Michael Waterman June 4, 2026



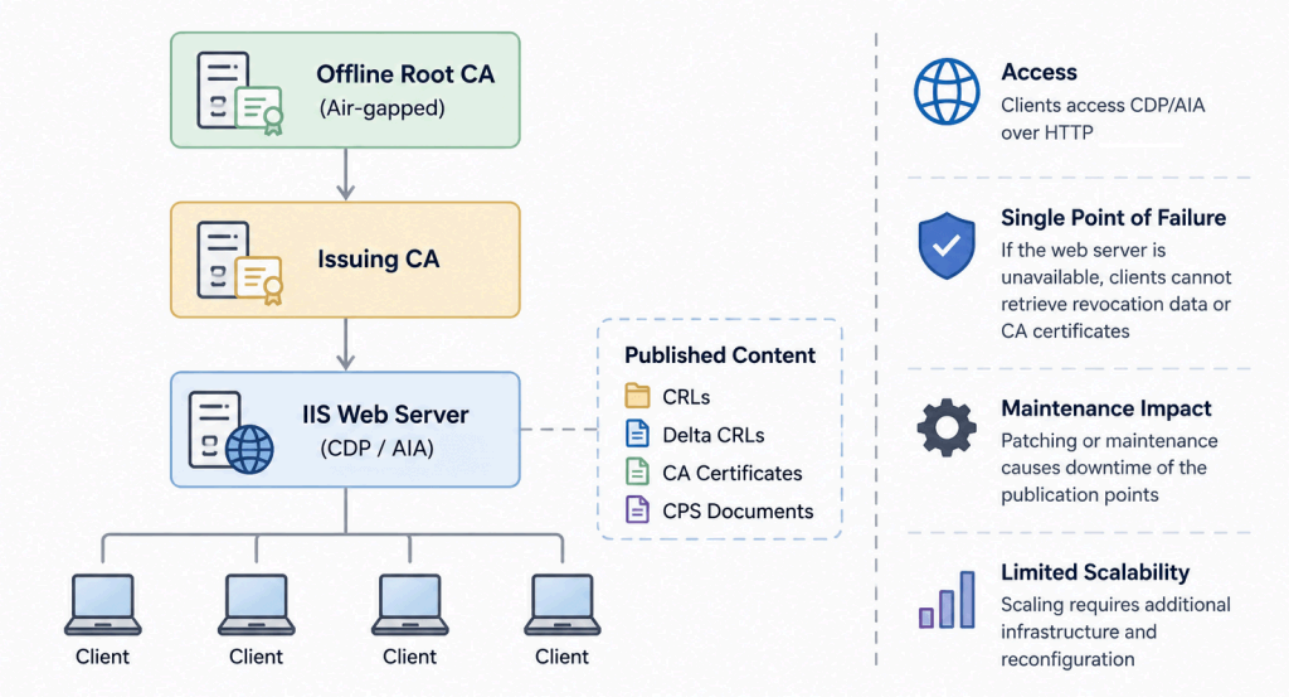
Last time I wrote about the why a Certificate Revocation List (CRL) should be available for the majority of services that make use of certificates. One of those prime examples is the use of smartcards. When revocation can't be checked, you simply can not logon. Most Microsoft PKI deployments start with a single web server hosting the CRL Distribution Point (CDP) and Authority Information Access (AIA) locations. While this works well for smaller environments or labs, it introduces a single point of failure. If the web server becomes unavailable, certificate revocation checking may fail and certificate validation can be disrupted across the environment.

Designing an building a high available setup can be difficult, you need to understand how it works and why it works in a certain way. In this article, I will build a highly available publication platform for CRLs, delta CRLs, CA certificates and CPS documents using DFS Namespaces, DFS Replication, IIS and Group Managed Service Accounts (gMSA). Along the way I will explain what I'm doing and why.

Let's dive in!

The single server problem

Let's first discuss what the problem actually is and why you would want such a solution in the first place. Many Active Directory Certificate Services (AD CS) deployments start with a simple design, a single web server hosting the publication points for CRLs, delta CRLs, CA certificates, and CPS documents. For smaller environments this approach is often sufficient and easy to manage. However, as the PKI becomes increasingly important to business operations, the availability of these publication points becomes more critical.



Single server setup

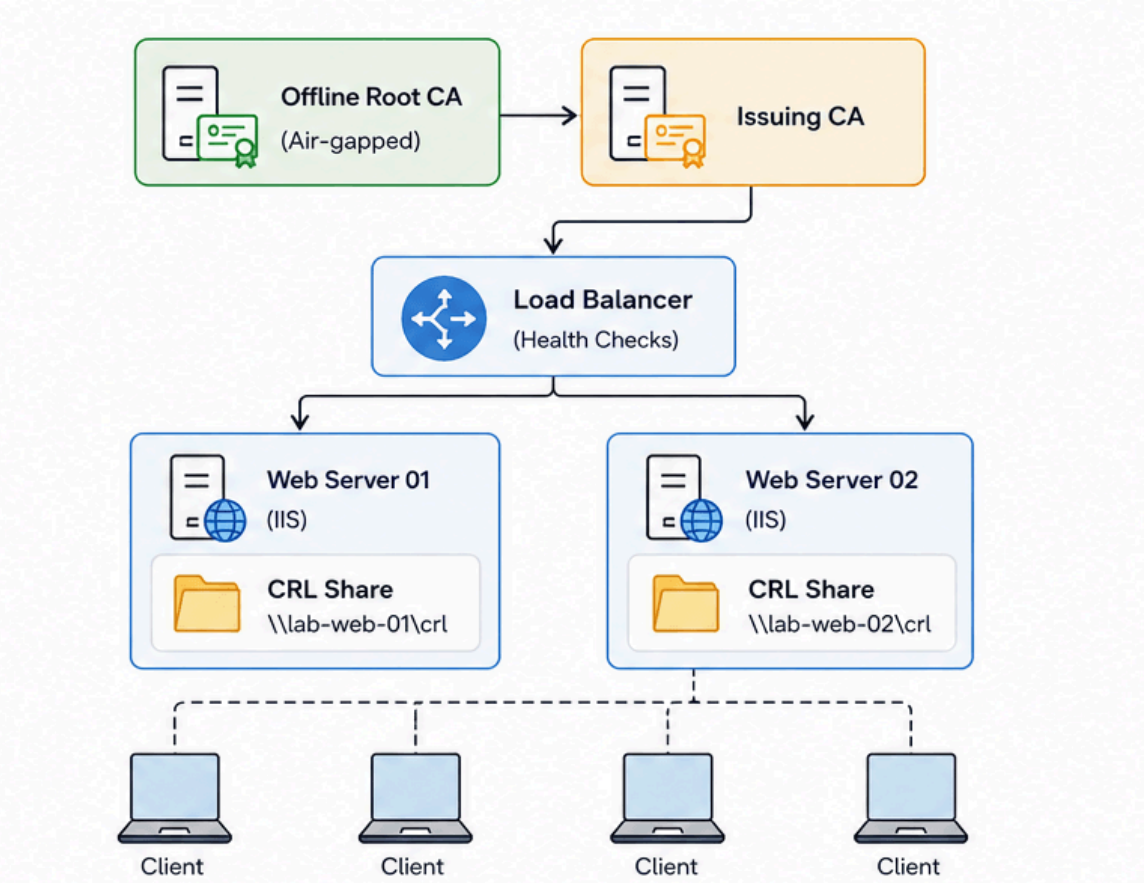
The challenge is that a single web server introduces a single point of failure. Planned maintenance, operating system updates, hardware failures, storage issues (just to name a few), or even a simple configuration mistake can make the publication platform unavailable. While existing CRLs may remain valid for some time, the inability to publish updated revocation information or distribute CA certificates can eventually impact certificate validation throughout the environment.

High availability is often considered for other critical infrastructure components. The publication platform, however, is frequently overlooked despite being an essential part of the trust chain. If clients cannot access revocation information when it is required, the availability of the PKI as a whole can be affected, as I've discussed in my previous blog post.

The goal of this blog is to eliminate the single point of failure while keeping the solution relatively simple and based entirely on native Windows Server components.

A simpler high availability model

Before diving into the full high availability solution, it is worth considering a much simpler design. In this model, two IIS web servers are deployed behind a load balancer. Each web server hosts a local CRL share that is used both for publishing CRLs and delta CRLs and for serving the published content to clients.



Components Overview

Component	Description
Offline Root CA	Trust anchor of the PKI. Kept offline and used only for signing subordinate CA certificates.
Issuing CA	Issues and revokes certificates. Publishes Base CRL and Delta CRLs to the web servers.
Load Balancer	Distributes client requests across the web servers and performs health checks.
Web Servers (2x)	IIS servers hosting a local CRL share. Serve CRL and Delta CRL files to clients.
Clients	Retrieve CRLs from the publication URLs via the load balanced DNS name.

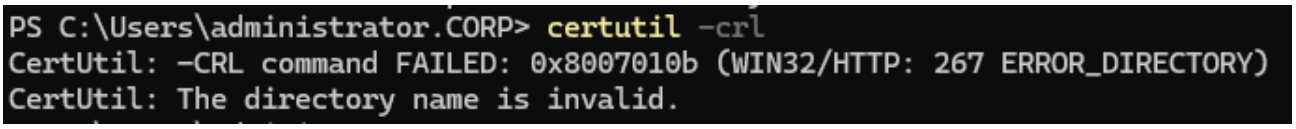
A simple high availability model

From a client perspective, this approach works well. Clients access a single DNS name, such as *pki.trust.corp.michaelwaterman.nl*, while the load balancer distributes requests

across both web servers. As long as the load balancer performs proper health checks, a failure of one web server remains transparent to clients and certificate validation continues to function normally.

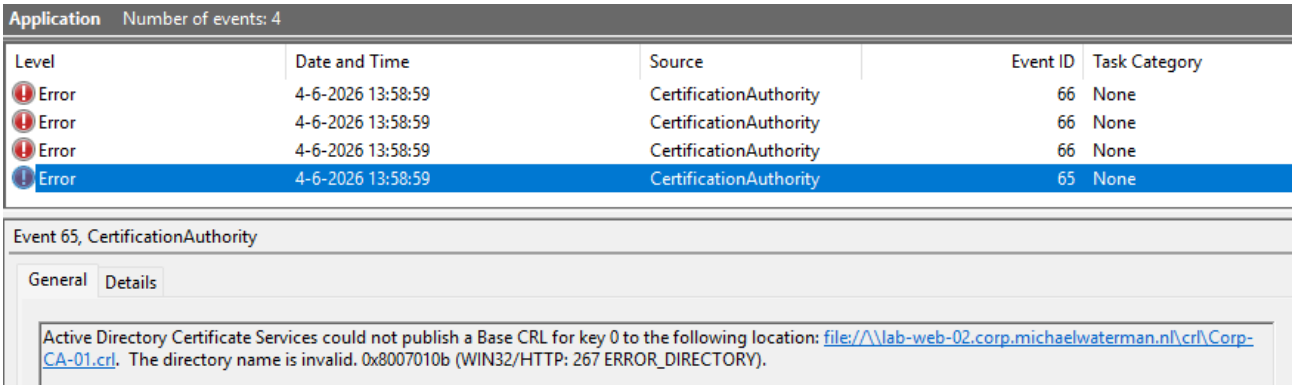
The challenge appears on the publishing side. The Issuing CA is configured to publish CRLs and delta CRLs directly to the shares hosted on the individual web servers. If one of these shares becomes unavailable due to a server outage, maintenance activity, or storage issue, the publication process can fail.

During my testing, the Base CRL continued to publish successfully to the remaining target, but delta CRL publication failed entirely. As a result, revocation checking could eventually be impacted despite the publication platform appearing highly available from the client perspective. During manual publishing with "*certutil -crl*", a timeout error appeared on the screen,



CRL Publishing error

and the following events were recorded on the Issuing CA:



Multiple errors while publishing

```
Active Directory Certificate Services could not publish a Base CRL for key 0 to the
following location:
file://\\lab-web-02.corp.michaelwaterman.nl\crl\Corp-CA-01.crl.
The directory name is invalid. 0x8007010b (WIN32/HTTP: 267 ERROR_DIRECTORY).

Active Directory Certificate Services could not publish a Delta CRL for key 0 to
the following location:
C:\WINDOWS\System32\CertSrv\CertEnroll\Corp-CA-01+.crl.
Operation aborted 0x80004004 (-2147467260 E_ABORT).

Active Directory Certificate Services could not publish a Delta CRL for key 0 to
the following location:
file://\\lab-web-01.corp.michaelwaterman.nl\crl\Corp-CA-01+.crl.
Operation aborted 0x80004004 (-2147467260 E_ABORT).
```

```
Active Directory Certificate Services could not publish a Delta CRL for key 0 to
the following location:
file://\\lab-web-02.corp.michaelwaterman.nl\crl\Corp-CA-01+.crl.
Operation aborted 0x80004004 (-2147467260 E_ABORT).
```

This does not necessarily make the design unusable. In smaller environments it can be a perfectly acceptable solution, provided that monitoring is in place and administrators understand the operational dependency on the publication shares. Maintenance activities should be carefully planned, and any outage affecting a publication target should be addressed before the next CRL publication cycle. You need to know in advance when the CRL or delta CRL will be published before rebooting the server.

One possible workaround is to temporarily disable the "*Publish CRLs to this location*" and "*Publish Delta CRLs to this location*" options for the unavailable target and restart the Certificate Services service. While this allows publication to continue, it also introduces operational overhead and manual intervention during outages.

Although simple and easy to deploy, this design ultimately demonstrates why a shared publication platform becomes valuable. By separating the publication process from the web servers themselves, we can eliminate these dependencies and create a more resilient architecture.

High availability design goals

Before designing a solution, it is important to define what we are trying to achieve. The goal is not to build a complex enterprise-grade publishing platform with expensive load balancers or third-party software. Instead, the objective is to improve the availability of the publication points while keeping the design relatively simple and easy to maintain, but with enterprise-grade quality in mind.

The platform should eliminate the single point of failure introduced by a standalone web server and allow maintenance to be performed without interrupting access to CRLs, delta CRLs, CA certificates, or CPS documents. At the same time, the solution should rely on native Windows Server components wherever possible, reducing both operational complexity and licensing costs.

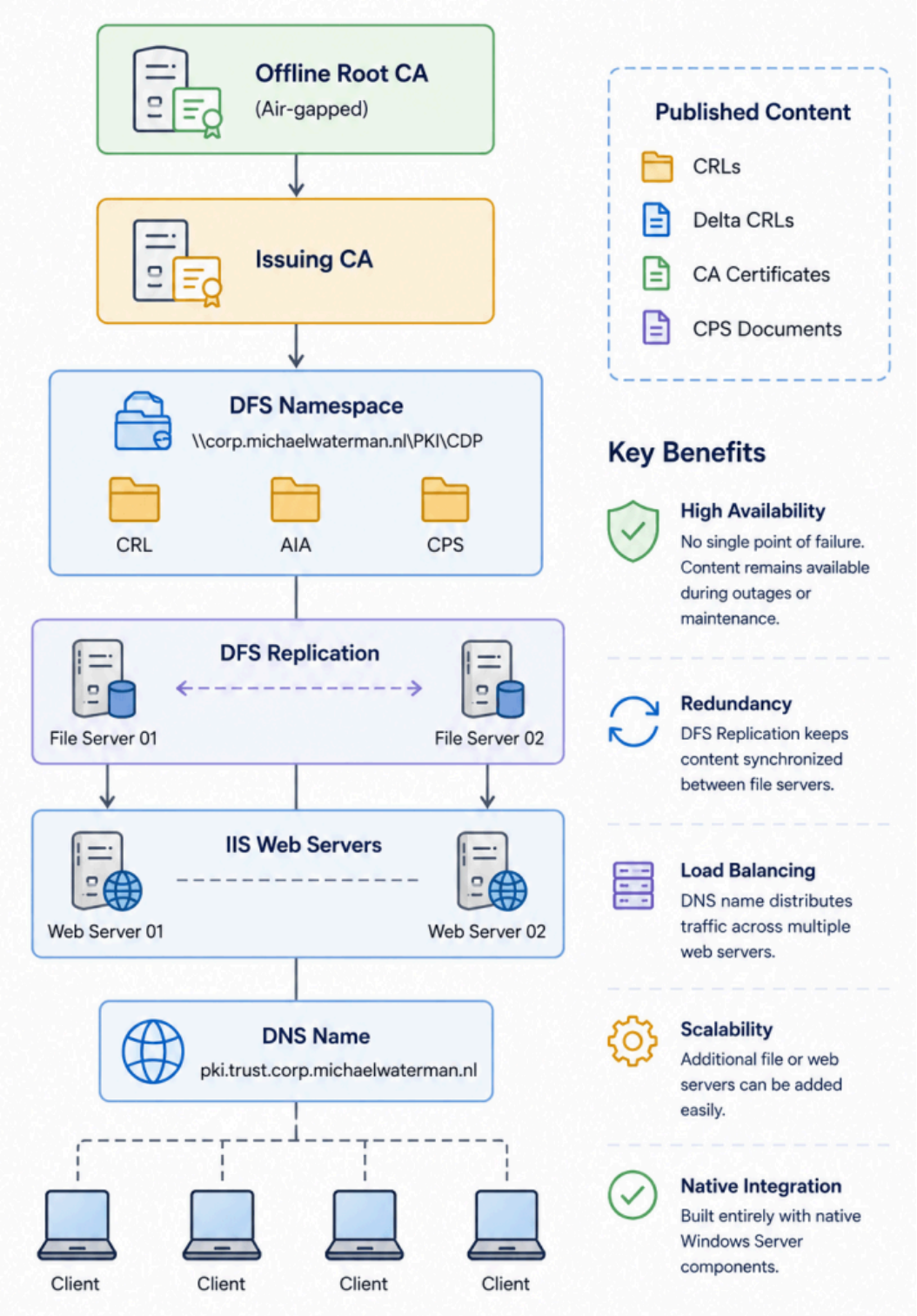
Another important requirement is centralized content management. Administrators should only have a single location where publication data is stored, while the web servers remain responsible solely for serving content to clients. This separation simplifies management and reduces the risk of configuration drift between servers.

Finally, the design should be scalable. While the initial deployment may consist of only two file servers and two web servers, the architecture should allow additional servers to be added in the future without requiring changes to the certificate authority configuration or published URLs.

Solution overview

In this chapter, I will show you the design for a highly available publication platform using native Windows Server components. By combining DFS Namespaces, DFS Replication, IIS, and Group Managed Service Accounts, I will eliminate this dependency on a single server while maintaining a centralized and easy-to-manage publication process.

The resulting architecture provides redundancy for both storage and web services, allowing the publication platform to remain available during maintenance activities, hardware failures, or server outages.



High-availability architecture overview

Solution components

Offline Root CA

The Offline Root CA serves as the trust anchor for the PKI hierarchy. It remains disconnected from the network and is only used for infrequent administrative tasks such as signing subordinate CA certificates and publishing CRLs.

Issuing CA

The Issuing CA is responsible for issuing and revoking certificates within the environment. It publishes CRLs, delta CRLs, and CA certificates to the publication platform.

DFS namespace

The DFS Namespace provides a single, consistent publication path that can be used by administrators and certification authorities. This abstraction layer removes dependencies on individual file servers.

DFS replication

DFS Replication synchronizes the published content between multiple file servers. This ensures that CRLs, CA certificates, and CPS documents remain available even if a file server becomes unavailable.

File servers

The file servers host the publication data used by the DFS Namespace. By using multiple servers, the storage layer becomes resilient against hardware failures and maintenance activities.

IIS web servers

The IIS web servers provide HTTP access to the published PKI content. Multiple web servers can be deployed to eliminate the web layer as a single point of failure.

Group managed service account (gMSA)

The gMSA provides a secure identity for the IIS application pools. It enables access to the publication share without requiring administrators to manage passwords manually.

DNS name

A single DNS name provides a consistent URL for clients accessing the publication platform. This allows web servers to be added, replaced, or load balanced without changing published certificate URLs. Obviously this can point to a hardware based load-balancer.

Clients

Clients (or servers) retrieve CRLs, CA certificates, and CPS documents from the publication platform during certificate validation. They remain unaware of the underlying redundancy and always use the same published URLs.

Before we start building the solution, it is worth mentioning that every configuration step in this article will be performed using PowerShell (because why not). While the graphical tools work perfectly fine, PowerShell provides a repeatable deployment process, makes documentation easier, and allows the entire environment to be recreated whenever needed.

More importantly, if you're reading a PKI blog, there is a reasonable chance that you enjoy PowerShell as much as I do. And if not, there is no better opportunity to start!

Requirements

Before building the solution, a number of prerequisites must be in place. This guide assumes that an Active Directory Certificate Services (AD CS) deployment already exists and that certificate publication is currently functional. The focus of this article is not the deployment of the certification authorities themselves, but rather the design and implementation of a highly available publication platform.

Active Directory

The environment must have an Active Directory domain with at least one writable domain controller. In addition, the DFS Namespace and DFS Replication features must be available, as these components form the foundation of the publication platform. The solution described in this article uses the following Active Directory domain:

> *corp.michaelwaterman.nl*

Certification authorities

The following certification authorities are assumed to be available:

- Offline Root CA
- Issuing CA

The Issuing CA must already be configured to publish CRLs, delta CRLs, and CA certificates. These publication locations will be updated later in the guide to point to the new highly available platform.

File servers

Two file servers are used to host the publication content and provide redundancy through DFS Replication.

- *lab-file-01.corp.michaelwaterman.nl*
- *lab-file-02.corp.michaelwaterman.nl*

These servers will host:

- DFS Namespace Root
- DFS Replicated Content
- SMB Shares

Web servers

Two IIS servers are used to publish the PKI content over HTTP.

- *lab-web-01.corp.michaelwaterman.nl*
- *lab-web-02.corp.michaelwaterman.nl*

The web servers will retrieve publication content from the DFS Namespace and make it available through a single DNS name.

DNS

A dedicated DNS name should be created for the publication platform. This ensures that the published URLs remain unchanged even if individual web servers are replaced or additional servers are added in the future. The following DNS name is used throughout this article:

> *pki.trust.corp.michaelwaterman.nl*

For lab environments, DNS Round Robin can be used to distribute requests between multiple web servers. Production environments will benefit from a dedicated load balancer capable of performing health checks as DNS lacks that feature.

Group managed service account

The IIS application pools will run under a Group Managed Service Account (gMSA). This account provides secure access to the DFS Namespace without requiring administrators to manage service account passwords. The name of the account will be:

> *gmsa-pkiweb*

Before creating the gMSA, ensure that the Key Distribution Service (KDS) has been initialized within the Active Directory forest.

Required roles and features

The following Windows Server roles and features are required:

- Active Directory Certificate Services (existing deployment)
- DFS Namespaces

- DFS Replication
- Web Server (IIS)
- Group Managed Service Accounts

Administrative permissions

The deployment steps in this guide require administrative permissions on the certification authority, file servers, web servers, and Active Directory. Domain Administrator privileges are recommended during the initial deployment phase.

Building the solution

On your management workstation, a PAW is preferred, open an elevated PowerShell session. Follow along with the PowerShell commands.

Security group

A security group is used for the web servers, this group is used to give access to resources instead of individual objects.

```
New-ADGroup `
-Name "GG_PKI_WebServers" `
-GroupScope Global `
-GroupCategory Security `
-Path -Path 'OU=Groups,OU=Tier 0,OU=Management,DC=corp,DC=michaelwaterman,DC=nl
```

Add the web servers to the group:

```
Add-ADGroupMember `
  -Identity "GG_PKI_WebServers" `
  -Members "lab-web-01$","lab-web-02$"
```

Setup a gMSA

The group managed service account is used as the service account that IIS uses to connect to the DFS namespace to service the files. check the existence of a root key:

If you don’t have a key listed, use the command below to create a new one.

```
Add-KdsRootKey -EffectiveTime ((Get-Date).AddHours(-10))
```

Else, create the gMSA:

```
New-ADServiceAccount `
  -Name "gmsa-pkiweb" `
```

```
  -DNSHostName "gmsa-pkiweb.corp.michaelwaterman.nl" `
  -PrincipalsAllowedToRetrieveManagedPassword "GG_PKI_WebServers"
```

Installing the gMSA on both web servers

On both "*lab-web-01*" and "*lab-web-02*" install the gMSA account so it can be used to connect back to the DFS namespace via the Application pool.

Install the RSAT Tools

From an elevated PowerShell session, use the following commands:

```
Install-WindowsFeature RSAT--PowerShell
restart-computer
```

Note! We restart the computers for group membership of the "*GG_PKI_WebServers*" security group.

Install the GMSa

From an elevated PowerShell session, use the following commands:

```
Install-ADServiceAccount gmsa-pkiweb
```

Test the account:

```
Test-ADServiceAccount gmsa-pkiweb
```

Install and configure DFS on both file servers

On both "*lab-file-01*" and "*lab-file-02*" install and configure the DFS role.

Install the role

From an elevated PowerShell session, use the following commands:

```
Install-WindowsFeature FS-DFS-Namespace, FS-DFS-Replication -IncludeManagementTools
```

Note! When installing on a Windows core server you can skip the management tools.

Create the directory structures

```
New-Item -Path d:\dfsroots\pki -Type Directory -Force
New-Item -Path d:\pki\cdp\aia -Type Directory -Force
New-Item -Path d:\pki\cdp\crl -Type Directory -Force
New-Item -Path d:\pki\cdp\cps -Type Directory -Force
```

Note! We separate the pki files from the dfs directories, this separates the view that end point will see while browsing the shares.

Set NTFS permissions on the DFSRoot PKI directory

```
$Path = "D:\dfsroots\pki"
$ACL = Get-Acl $Path

# Disable inheritance and remove inherited entries
$ACL.SetAccessRuleProtection($true, $false)

# Remove all existing explicit ACEs
foreach ($Access in $ACL.Access) {
    $ACL.RemoveAccessRule($Access)
}

# SYSTEM
$ACL.AddAccessRule(
    (New-Object System.Security.AccessControl.FileSystemAccessRule(
        "SYSTEM",
        "FullControl",
        "ContainerInherit,ObjectInherit",
        "None",
        "Allow"
    ))
)

# Administrators
$ACL.AddAccessRule(
    (New-Object System.Security.AccessControl.FileSystemAccessRule(
        "Administrators",
        "FullControl",
        "ContainerInherit,ObjectInherit",
        "None",
        "Allow"
    ))
)

# Authenticated Users
$ACL.AddAccessRule(
    (New-Object System.Security.AccessControl.FileSystemAccessRule(
        "Authenticated Users",
        "ReadAndExecute",
        "ContainerInherit,ObjectInherit",
        "None",
        "Allow"
    ))
)

Set-Acl -Path $Path -AclObject $ACL
```

In the code above System and Administrators will have full control, Authenticated users will have the right to read and execute files.

Create the DFS Root share on *lab-file-01* & *lab-file-02*

```
New-SmbShare -Name "pki" `
  -Path "d:\dfsroots\pki" `
  -ReadAccess "Authenticated Users" `
  -FullAccess ".\Administrators" `
  -CachingMode None
```

This will provide read access to the group “Authenticated users” and full access to Admins from a file share perspective.

Create the DFS Namespace on *lab-file-01* and add *lab-file-02*

The following PowerShell commands need to be executed just once from either of the file servers or your management station.

```
New-DfsnRoot `
  -TargetPath "\\lab-file-01.corp.michaelwaterman.nl\pki" `
  -Type DomainV2 `
  -Path "\\corp.michaelwaterman.nl\pki" `
  -Description "PKI publication namespace"

New-DfsnRootTarget `
-Path "\\corp.michaelwaterman.nl\pki" `
-TargetPath "\\lab-file-02.corp.michaelwaterman.nl\pki"
```

I’m deliberately using FQDN’s of the servers here, during my testing I ran into issues when using just the hostname (Yes it’s always DNS)

Create the file share on *lab-file-01* & *lab-file-02*

On both file servers you will need to create the file shares. The “*Cert Publishers*” group is granted full access as the objects in that group (The Certificate Authorities) will need to be able to write CRL’s to that location.

```
New-SmbShare -Name "cdp" `
  -Path "d:\pki\cdp" `
  -ReadAccess "Authenticated Users" `
  -FullAccess "Cert Publishers", ".\Administrators" `
  -CachingMode None
```

Set the NTFS permissions on the CRL directory

As the members of the "*Cert Publishers*" will only need to write the CRL files, you will need to adjust the NTFS permissions first.

```
$ACL = Get-Acl "d:\pki\cdp\crl"
$AccessRule = New-Object System.Security.AccessControl.FileSystemAccessRule `
  ("Cert Publishers","FullControl","ContainerInherit,ObjectInherit","None","Allow")
$ACL.SetAccessRule($AccessRule)
Set-Acl -Path "d:\pki\cdp\crl" -AclObject $ACL
```

Add shares to the DFS namespace

After I've created the individual shares on both the file servers, I need to create and add them to the DFS namespace.

```
New-DfsnFolder `
  -Path "\\corp.michaelwaterman.nl\pki\cdp" `
  -TargetPath "\\lab-file-01.corp.michaelwaterman.nl\cdp"

New-DfsnFolderTarget `
  -Path "\\corp.michaelwaterman.nl\pki\cdp" `
  -TargetPath "\\lab-file-02.corp.michaelwaterman.nl\cdp"
```

Configure directory replication

Now that the file shares have been created and added to the DFS namespace, we need to configure it for replication. The following commands should only be executed once for one of the file servers or your management station.

Create the replication group

```
New-DfsReplicationGroup -GroupName "RG-PKI-CDP"
```

Add the file servers to the DFS replication group

```
Add-DfsrMember -GroupName "RG-PKI-CDP" -ComputerName "lab-file-
01.corp.michaelwaterman.nl"
Add-DfsrMember -GroupName "RG-PKI-CDP" -ComputerName "lab-file-
02.corp.michaelwaterman.nl"
```

Setup the replication connection

```
Add-DfsrConnection `
  -GroupName "RG-PKI-CDP" `
  -SourceComputerName "lab-file-01.corp.michaelwaterman.nl" `
  -DestinationComputerName "lab-file-02.corp.michaelwaterman.nl"
```

Define the DFS replication directory

```
New-DfsReplicatedFolder `
  -GroupName "RG-PKI-CDP" `
  -FolderName "CDP"
```

Configure the replication of the physical directories

```
Set-DfsrMembership `
  -GroupName "RG-PKI-CDP" `
  -FolderName "CDP" `
  -ComputerName "lab-file-01.corp.michaelwaterman.nl" `
  -ContentPath "d:\pki\cdp" `
  -PrimaryMember $true `
  -Force

  Set-DfsrMembership `
  -GroupName "RG-PKI-CDP" `
  -FolderName "CDP" `
  -ComputerName "lab-file-02.corp.michaelwaterman.nl" `
  -ContentPath "d:\pki\cdp" `
  -Force
```

From this point onwards, DFS replication will commence. In my experience this can take a couple of minutes, so be patient, grab a cup of coffee and check replication using the troubleshooting steps below…. just in case.

Create an example CPS (from lab-file-01)

To test and see replication in action, I created the certificate policy file. This file needs to exist anyway, and what better way to have it available on all file servers and see the DFS replication in action.

```
New-Item -Path "\\corp.michaelwaterman.nl\pki\cdp\cps\cps.html" -ItemType File
Set-Content -Path "\\corp.michaelwaterman.nl\pki\cdp\cps\cps.html" -Value
'<html>Placeholder for the Certification Practice Statement</html>'
```

Troubleshooting DFS replication

Here are some of the command I used during troubleshooting. Basically it came down to me not waiting long enough before replication kicked in. Patience is apparently not my forte

```
dfsrdiag backlog /rgname:RG-PKI-CDP /rfname:CDP /smem:lab-file-01 /rmem:lab-file-02
dfsrdiag backlog /rgname:RG-PKI-CDP /rfname:CDP /smem:lab-file-02 /rmem:lab-file-01

dfsrdiag pollad
```

```
Get-DfsrMembership -GroupName "RG-PKI-CDP"

Get-DfsrMembership -GroupName "RG-PKI-CDP" |
Select-Object
GroupName,FolderName,ComputerName,ContentPath,Enabled,PrimaryMember,ReadOnly
```

Installing and configuring the web servers

On both “*lab-web-01*” and “*lab-web-02*” install and configure the IIS Web Server role. The commands demonstrated below should be executed on *both* web servers.

Install the role

From an elevated PowerShell session, use the following commands:

```
Install-WindowsFeature Web-Server -IncludeManagementTools-IncludeManagementTools
```

Note! When installing on a Windows core server you can skip the management tools.

Create the directory structures

```
New-Item -Path "C:\inetpub\cdp\aia" -Type Directory -Force
New-Item -Path "C:\inetpub\cdp\cps" -Type Directory -Force
New-Item -Path "C:\inetpub\cdp\crl" -Type Directory -Force
```

Note! These directories will not acutally be used but will serve as redirection to the DFS namespace.

Create the web site

```
New-Website `
  -Name "Certificate Distribution Point" `
  -PhysicalPath "C:\inetpub\cdp" `
  -HostHeader "pki.trust.corp.michaelwaterman.nl"
```

Create the virtual directories (optional)

```
New-WebVirtualDirectory `
  -Site "Certificate Distribution Point" `
  -Name "aia" `
  -PhysicalPath "\\corp.michaelwaterman.nl\pki\cdp\aia" `
  -Force

New-WebVirtualDirectory `
  -Site "Certificate Distribution Point" `
  -Name "cps" `
  -PhysicalPath "\\corp.michaelwaterman.nl\pki\cdp\cps" `
```

```
  -Force

New-WebVirtualDirectory `
  -Site "Certificate Distribution Point" `
  -Name "crl" `
  -PhysicalPath "\\corp.michaelwaterman.nl\pki\cdp\crl" `
  -Force
```

Note! creating virtual directories is optional, it's just something that I like to do.

Configure IIS AppPool

```
Import-Module WebAdministration

New-WebAppPool -Name "PKI-CDP"

Set-ItemProperty "IIS:\AppPools\PKI-CDP" `
-Name processModel.identityType `
-Value 3

Set-ItemProperty "IIS:\AppPools\PKI-CDP" `
-Name processModel.userName `
-Value "CORP\gmsa-pkiweb$"

Set-ItemProperty "IIS:\AppPools\PKI-CDP" `
-Name processModel.password `
-Value ""
```

Note! As you can see in the code above, this is where we use the gMSA account. This account will be making the connections to the DFS/ SMB shares.

Attach the AppPool to the web site

```
Set-ItemProperty "IIS:\Sites\Certificate Distribution Point" -Name applicationPool
-Value "PKI-CDP"
```

Allow "double escaping" in IIS request filtering

```
Set-WebConfigurationProperty `
-PSPath "IIS:\Sites\Certificate Distribution Point\" `
-Filter "system.webServer/security/requestFiltering" `
-Name "allowDoubleEscaping" `
-Value $true
```

Set client side caching for static content

```
Set-WebConfigurationProperty -Filter "system.webServer/staticContent/clientCache" `
  -Name "cacheControlMode" -Value "UseMaxAge" -PSPath "IIS:\sites\Certificate
```

```
Distribution Point"

Set-WebConfigurationProperty -Filter "system.webServer/staticContent/clientCache" `
  -Name "cacheControlMaxAge" -Value "7.00:00:00" -PSPath "IIS:\sites\Certificate
Distribution Point"

Set-WebConfigurationProperty -Filter "system.webServer/staticContent/clientCache" `
  -Name "setEtag" -Value $true -PSPath "IIS:\sites\Certificate Distribution Point"
```

Set the AIA and CRL directory browsable (Optional)

```
Set-WebConfigurationProperty `
  -PSPath "IIS:\Sites\Certificate Distribution Point" `
  -Filter "/system.webServer/directoryBrowse" `
  -Name enabled `
  -Value $true
```

While there's no secret information within the directories published by the web server, it's recommended not to enable content browsing from the web. However this can be a useful feature during troubleshooting or a lab setup.

Unlock configuration access

```
& "$env:windir\system32\inetsrv\appcmd.exe" unlock config `
  /section:system.webServer/security/authentication/anonymousAuthentication
```

Some settings in IIS are locked at the root level. Allowing anonymous access is one of those features that should be unlocked for change first before alteration on the web site level is possbile, hence the unlock above.

Set anonymous access

```
Set-WebConfigurationProperty `
  -PSPath "IIS:\Sites\Certificate Distribution Point" `
  -Filter "/system.webServer/security/authentication/anonymousAuthentication" `
  -Name "userName" `
  -Value ""
```

Check with:

```
Get-WebConfiguration `
  -Filter "/system.webServer/security/authentication/anonymousAuthentication" `
  -PSPath "IIS:\Sites\Certificate Distribution Point" |
Select-Object enabled,userName,password
```

Publish the PKI Beacon

```
"PKI Beacon" | Out-File "C:\inetpub\cdp\beacon.txt" -Encoding ascii
```

Reset the IIS web server

Note! This article focuses on the publication platform itself and therefore does not cover load balancer deployment or configuration in detail. Any load balancer capable of performing HTTP health checks can be used in front of the IIS web servers. Readers interested in implementing a software-based load balancer may find one of my previous articles useful:

Building High Available LDAPS Architectures

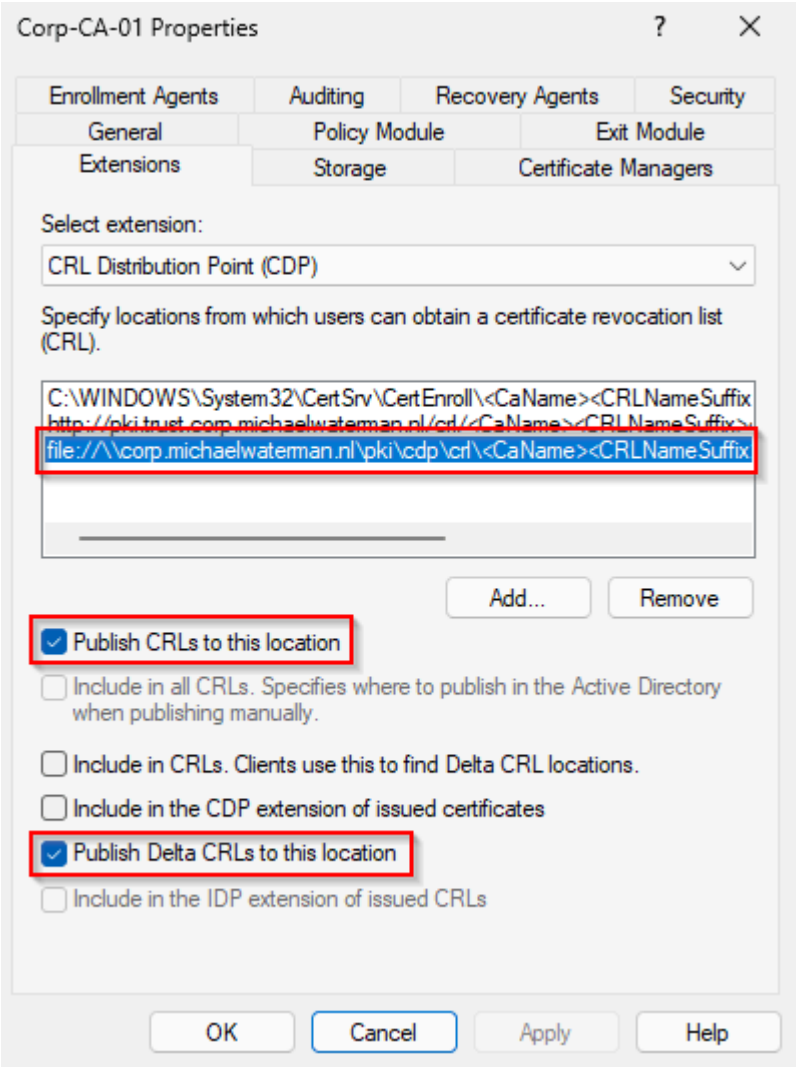
Although the article focuses on LDAPS, many of the same design principles apply to highly available PKI publication services.

Configure AD CS publication locations

With the highly available publication platform in place, the final step is to configure the Certification Authority to publish CRLs, delta CRLs, and CA certificates to the DFS Namespace.

The Issuing CA should be configured to publish its content to the DFS Namespace rather than directly to individual web servers. This ensures that publication remains independent of the web tier and allows DFS Replication to distribute the content across the file servers automatically. In this implementation, the Certification Authority publishes to the following DFS paths:

> *corp.michaelwaterman.nl**PKI**CDP**crl*



DFS locations added to the publishing location

Once the publication locations have been configured, publish a new Base CRL and verify that the files appear within the DFS Namespace. After replication has completed, confirm that the content is accessible through both IIS web servers and through the public DNS name.

Finally, verify that certificate validation succeeds and that CRLs, delta CRLs, and CA certificates can be retrieved from the published URLs. At this point, the publication platform is fully operational and capable of tolerating file server or web server outages without requiring changes to the Certification Authority configuration. For example, log in as a regular user, open a PowerShell session and use the following to check the PKI beacon:

```
Invoke-WebRequest -UseBasicParsing -Uri
http://pki.trust.corp.michaelwaterman.nl/beacon.txt
```

The result should be "*PKI Beacon*", as we configured during the setup.

After publishing the CRLs and CA certificates, it is recommended to perform an end-to-end validation. While manually browsing to the published URLs confirms that the files are available, it does not verify that certificate chain building and revocation checking function

correctly. One of the easiest validation methods is to use `certutil.exe` with the `-urlfetch` and `-verify` options (or simply use `certutil -url <cert.cer>` for a gui based validation). This forces the client to retrieve the AIA and CDP locations and validates that the published content can be used successfully during certificate validation.

```
certutil -urlfetch -verify <certificate.cer>
```

Successful validation confirms that the publication platform, certificate chain, CRLs, and delta CRLs are all functioning as expected.

Final thoughts

When designing a Public Key Infrastructure, attention is often focused on the Certification Authorities themselves. While protecting the Root CA and Issuing CA is very important, the publication platform that distributes CRLs and CA certificates is equally important. Without reliable access to this information, certificate validation can quickly become problematic.

Throughout this article, I explored both a simple load-balanced design and a more resilient architecture based on DFS Namespaces, DFS Replication, IIS, and Group Managed Service Accounts. While the simpler approach can be suitable for smaller environments, it introduces operational dependencies that become increasingly difficult to manage as availability requirements grow.

By separating publication, storage, replication, and web services into dedicated components, I created a platform that is easier to maintain, easier to scale, doesn't cost anything extra (well, almost) and is far more resilient to outages. Perhaps most importantly, the solution is built entirely on native Windows Server components, making it accessible to any organization already running Active Directory Certificate Services.

High availability does not always require complex solutions. Sometimes it simply requires placing the right components in the right place and use what you already have anyway.

As always, I hope this post has been informative and useful to your setup. Until next time!